

Albertitos plan

An agentic-first website for paying suppliers

Team	la_interseccion
teamId	n571h527bt7jppe6azj9bpbcejn8em1c9
Members	Giancarlo Buonomo · Liam Szefner · Javier Gismero · Miquel Berenguer Flores
Demo	hackspain.vercel.app
Delivery	20 September 2026, Madrid



la_interseccion, hour 20

What it is

Alberto is the person who pays the invoices. Albertito is what we built for him. It reads the 500 invoices in La Caja, returns PAGAR, NO_PAGAR or ESCALAR for each one, and can show why on any of them.

Alberto does not open a terminal. He opens a website that behaves like an agent: he asks a question and gets back a working answer, not a report. Under it runs a durable engine on Postgres. That engine makes the decisions and writes the delivered file.

One idea holds up the rest

A model reads the paper. A second model can challenge the verdict afterwards. Neither one can authorise a payment. The verdict itself is code: the same invoice, the same rules and the same ERP snapshot always give the same answer.

Who does what

Step	Who	Why
Read the invoice	Code first, a vision model only when a field is missing	Most pages carry text. Every value points at the exact characters it came from
Compile the rules	A classifier over the spreadsheet, once	Free text becomes a rulebook you can read. It is cached, so nobody pays to compile it twice
Decide	Code. Six checks and one policy. No network	Auditable and repeatable, today and in January
Challenge it	A reasoning model, before any payment	It can stop a PAGAR. It can never create one
Resolve	Alberto	What is left is grouped, already investigated, and one click from the evidence

1. Architecture

1.1 The product

Good morning, Alberto.

I did the boring part while you slept. What is left needs an actual human, which is where you come in.

 Ask me anything, throw me a batch, or invent a new rule...

↑

Which payments do I need to do today

Which invoices do I need to review

Show me today's report

Throw a or a anywhere on this page

Everything that could be cleared overnight is already cleared. What is on screen is what needs a person.

We did not build a dashboard. Alberto is not exploring data, he is clearing a queue before lunch, so the page opens on a question box and the archive sits to one side. He asks, and the answer comes back as something he can act on: a group of invoices with its buttons, a file with its trace, a new rule with the backtest already run.

Nothing is stored twice. The system keeps an append-only log of what happened, and the current state is read from it. That is where the trace, the cost per invoice and the closing report all come from. It is also why one invoice can show two verdicts at once, one per rulebook version, without either of them being overwritten.

1.2 Six checks, in order

Six checks run on every invoice, always in the same order. The first one that stops it decides, and nothing outside the list has a say. READABLE is a gate. DUPLICATES is the only check that refuses a payment outright. The rest hand the invoice to Alberto instead of rejecting it.

Rules

Six checks run on every invoice, in this order. The first one that stops it decides, and nothing outside this list has a say. Change one and every verdict can be replayed against it.

v3.2

sha256 9f2c1ab4

500 checked · 54 stopped

1	READABLE	The page has a text layer at all.	GATE		35	>
2	DUPLICATES	No invoice with this number has been paid before.	DO NOT PAY		2	>
3	VENDOR	The account and the NIF on the page match the master.	ESCALATE		8	>
4	AMOUNT	Base plus VAT reaches the total the page states.	ESCALATE		6	>
5	DATES	There is a readable issue date, and the terms are known.	ESCALATE		3	>
6	MISSING	A purchase order number is on the page.	ESCALATE		0	>

WHAT CHANGED · 2

v3.1 → v3.2	VAT tolerance raised to 0.5% <i>you said: "round the cents, a two cent gap is not a problem"</i>	18 Sep 14:20
v3.0 → v3.1	DUPLICATES now blocks instead of escalating <i>you said: "if we already paid it, do not even ask me"</i>	16 Sep 09:05

The rules page is built from the engine, not written by hand. Each row shows what that check stopped, and every change keeps the sentence that asked for it.

Changing a rule is a change to a file, not to the code. The rulebook carries a fingerprint of its own contents, so two versions can coexist and every past decision still points at the exact one that produced it.

1.3 The trace

One endpoint returns everything stored about an invoice, from the bytes that arrived to the verdict and the codes behind it. Those codes are the contract. When we changed engines the rules were renumbered, and anything that had matched on rule numbers would have kept answering confidently and wrongly.

Search a file, a supplier or an invoice number...

AllPay · 446Review · 52Do not pay · 2500 of 500

> Ofimática Cieza S.L. 48

PAY€261,531.3746 invoicesNEEDS YOU€8,352.002 invoicesNO PAY€0.000 invoices

▼ Construcciones Benimaclet S.A. 8

PAY€115,765.237 invoicesNEEDS YOU€7,502.001 invoiceNO PAY€0.000 invoices

FILE	INVOICE	ISSUED	AMOUNT	ACTION	WHAT BLOCKED IT
FA-1123_construcciones.pdf	FA-1123	—	€7,502.00	ESCALATE	dates
2026-01-24_P009.pdf	2026/25704	24 Jan	€2,784.62	PAY	—
2026-05-24_P009.pdf	2026/45002	24 May	€2,852.45	PAY	—
2026-06-16_P009.pdf	2026/24305	16 Jun	€6,076.67	PAY	—
2026-07-01_P009.pdf	2026/38406	01 Jul	€84,700.00	PAY	—
2026-23904_construcciones.pdf	2026/23904	05 Jun	€6,195.20	PAY	—
2026-87003_construcciones.pdf	2026/87003	14 Feb	€3,758.01	PAY	—
factura_7835.pdf	2026/23601	12 Mar	€9,398.28	PAY	—

> Limpiezas Turia S.L. 44

PAY€191,127.4942 invoicesNEEDS YOU€7,030.102 invoicesNO PAY€0.000 invoices

The archive, grouped by supplier. The last column is the one that matters: what stopped each invoice, in the engine's own words.

The line for minute five

Every decision points at the check that made it, at the characters on the page the numbers came from, and at the ERP entry behind it. Nothing on screen is recalculated by hand.

1.4 When something breaks

The 2009 ERP bridge fails in three documented ways and all three are real. We retry the page that failed, log in again when the session dies, and wait when it asks us to. Before trusting a download we check the total the ERP itself declares, because a snapshot that is quietly incomplete is worse than an error.

Provider failures cost time, never a wrong payment. Work is durable: finished jobs are reused, a failure records the attempt instead of half a row, and relaunching continues where it stopped. If the vision model is down, everything the text layer can read is still decided. If the reviewer is down, nothing is paid and the invoice goes to Alberto.

1.5 Speed and cost

Measurement	Value	Conditions
One invoice, warm	4.2 s	Text layer, rules and ERP snapshot reused, everything committed
One invoice, cold	10.2 s	Fresh ERP capture and first publication of the rules cache
Vision	1 request per page	Down from a layout pass, two reads and a comparison
Reviews	3 at a time	They overlap behind the evaluations, which run in order

Measurement	Value	Conditions
ERP snapshot reuse	30 s	Complete captures only. Partial ones are never cached
Model calls per invoice	0, 1 or 2	None to decide it. One review before a payment, plus one vision request per page if the text layer fails
Cost of the review	2.5 cents	Worst case: both configured ceilings hit, at the untariffed rate we overestimate with
Cost of a vision page	0.6 cents	Same ceilings at the reader's rate. Only for pages without a text layer
Worst case for the 500	under 16 euros	Every invoice reviewed and every page also falling through to vision

Most invoices cost nothing to decide. The text layer reads them, the six checks run locally, and no model is called at all. Money is only spent in two places: a vision call on a page the text layer could not read, and the review that runs before a payment. Both record the tokens the provider reports, per invoice, in the same log as everything else.

Those cents are a ceiling, not an average. We set both caps ourselves: 100,000 characters of input, roughly 25,000 tokens, and 8,192 tokens of output per call. At the rate we use for a model the gateway does not tariff, and we deliberately overestimate there, a review that hits both caps costs 2.5 cents. A vision page costs 0.6. Nothing can quietly exceed that, because oversized input fails loudly instead of being truncated.

So a run over the whole box has a worst case of under 16 euros, with every single invoice reviewed and every page also falling through to vision. The real bill is lower, and it is recorded rather than guessed.

The figure we charge to an invoice comes from the provider's own response, recorded per file. When the endpoint returns no cost, the invoice is marked unknown instead of estimated. The rates above are list prices we wrote down, not an invoice we have paid, and we would rather say that than put a made-up number on a slide and watch it end up in someone's budget.

2. ADRs and trade-offs

ADR-1 The verdict is code, not a model

Context	The rules arrive as free text in a spreadsheet called FINAL_v7_DEFINITIVO_ahorasi.xlsx. The obvious move is to hand a model the invoice and the rules and ask what to do.
Alternatives	An LLM judge on every invoice. An LLM that writes the rule code on every run. Or read the rules once, compile them, and decide in code.
Decision	The third. Compiling happens once, and the result is cached against the spreadsheet, the mapping and the profile it came from. Two invoices cannot pay to compile the same rules at the same time, and if the source changes the cache is refused instead of reused.
Consequences	Changing the rules means recompiling and reprocessing. A rule the classifier cannot place stays off until someone looks at it. And there is no intelligence at the moment of deciding, which is the point.
Evidence	Our first live run escalated everything for reasons no invoice could have avoided: a rule asked for a supplier status no source carries, the duplicate history declared itself incomplete so absence could never be proven, and one evaluation date was applied to invoices spanning six months. All three are fixed, and each fix sits next to the parameter it produced.

ADR-2 Read the text layer first, use vision only for the gaps

Context	For a while every PDF went through vision: a layout pass, two reads and a comparison. Most of La Caja has a text layer on every page.
Alternatives	Send everything to the model. Send whatever fails arithmetic. Or send only what is missing a field the rules need.
Decision	The last one. A deterministic stage reads the text layer and anchors each field to the label that opens its line. It is accepted only when every value the rules need is there and validation is green. Otherwise the gaps are recorded and vision runs. We rejected triggering on arithmetic: an invoice whose total does not add up is the finding, and asking a model to read it again is asking it to erase that.
Consequences	Two extraction paths to maintain. In exchange, a bug in the fast one costs a vision call, never the invoice.
Evidence	The census over La Caja is in the repo: which invoices carry text, which the fast path accepted, and the few it rejected for printing impossible dates like 30/02, which go to vision exactly as intended. Every outcome names the path that produced it.

ADR-3 Nothing is paid unreviewed, and review can never say yes

Context	Deterministic checks miss what only context catches: a pattern across invoices, something odd the rules were never written for.
Alternatives	Ship the checks as final. Let a reviewer overrule them both ways. Or let it block a payment but never approve one.
Decision	The third, and the asymmetry is deliberate. A PAGAR is delivered only if the review leaves it standing. An unreviewed PAGAR escalates. Review can be turned off for a fast run, and then the trace says so instead of pretending a review happened.
Consequences	Review was half the wall time, so three run at once. A provider outage costs throughput and a pile of escalations, never a wrong payment.
Evidence	The reviewer failed once and left nothing behind, because the raw response had been thrown away before anyone could read it. Failures now keep the raw bytes. That is how we found the model spending its entire budget thinking and returning an empty answer, and fixed it.

ADR-4 Reason codes are the contract, not rule numbers

Context	Rules get renumbered and sentences get rewritten. Ours were renumbered when the engine changed.
Alternatives	Let consumers match on the rule number, on the sentence, or on a stable code carried in the result.
Decision	The stable code. The website, the exports and the phone agent all match on codes. Anything keyed on rule numbers would have kept answering after the renumbering, and nobody would have noticed until the wrong thing was said out loud.
Consequences	Codes are a public surface. Adding one is cheap, changing what one means is not.
Evidence	The duplicate check also demanded a printed currency, which the configured key never asked for. One invoice whose template prints none made the history incomplete for every invoice after it and blocked the batch. Invisible per invoice, obvious in the codes.

ADR-5 Everything downstream reads decisions, it never makes them

Context	The website shows verdicts and the phone agent answers suppliers. Both are tempting places to recompute something.
Alternatives	Let each surface query the engine and work out what to show, or freeze what the engine already decided and read that.
Decision	Freeze and read. Earlier, a web backend imported a second engine and recalculated verdicts while the deliverable came from the first one, so an invoice could show one result on screen and another in the file. Now nothing on screen is recalculated.
Consequences	A surface can be out of date if its export is old, and we would rather say that than invent a number. The deployed site also refuses to claim actions it cannot perform: where it cannot move real state, it says it did not.
Evidence	Every route, status code and payment rule is written down, with real captured payloads and a typed client, so nobody downstream has to guess a shape.

3. Bonus: the phone agent

A supplier calls asking why he has not been paid. Somebody has to pick up, find the invoice, work out what happened and answer without saying anything they should not. We built that. Inés answers the phone, identifies the caller, asks for the order number, looks up the decision the engine already made, and explains it out loud.

Descolgar

Colgar

consultando alberto.db...

INÉS

Contabilidad de proveedores, buenos días. Le atiende Inés. ¿Me dice el nombre de su empresa?

PROVEEDOR

Electricidad Montcada

INÉS

Electricidad Montcada S.A., CIF A46990201. ¿Es correcto?

PROVEEDOR

sí

INÉS

Gracias. ¿De qué pedido me habla? El número PO-....

PROVEEDOR

el cuatro noventa y tres

INÉS

Factura 12.874,40 € frente a pedido 12.847,40 €: 27,00 € de diferencia.

Escribe lo que dice el proveedor...

Enviar

Hablar

► Suministros Levante · PO-2026-0475 · ya estaba pagada

Expediente

FACTURA
FA-5077_electricidad.pdf

PROVEEDOR
Electricidad Montcada S.A.

PEDIDO
PO-2026-0493

IMPORTE
12.874,40 €

ESCALAR

NIF_CONTROL_MISMATCH; UNUSABLE_INPUT; ORDER_AMOUNT_MISMATCH; PAYMENT_TERMS_EXCEEDED; UNREVIEWED_ANNOTATION; UNUSABLE_INPUT

VIOLATED R1

{ "supplier.id": "P006", "invoice.supplier_tax_id": "A46990201", "invoice.iban": "ES3520385778983000765410", "supplier.tax_id": "A46990201", "supplier.iban": "ES3520385778983000765410" }

PASS R2

{ "invoice.number": "FA-5077", "supplier.id": "P006", "invoice.total": "12874.4", "invoice.issue_date": "2026-06-14", "invoice.order_reference": "PO-2026-0493", "order.id": "PO-2026-0493", "order.supplier_id": "P006", "erp.order_payment_status": "PENDIENTE", "invoice.currency": "EUR" }

VIOLATED R3

{ "invoice.line_amounts": ["106.4", "1064", "9469.6"], "invoice.taxable_base": "10640", "invoice.vat_amount": "2234.4", "invoice.total": "12874.4", "invoice.currency": "EUR", "invoice.order_reference": "PO-2026-0493", "invoice.taxes.0.kind": "vat", "invoice.taxes.0.rate_percent": "21", "invoice.taxes.0.amount": "2234.4", "supplier.id": "P006", "order.supplier_id": "P006", "order.id": "PO-2026-0493", "order.total": "12847.4", "order.currency": "EUR" }

NEEDS_REVIEW

{ "invoice.issue_date": "2026-06-14", "supplier.id": "P006", "supplier.payment_terms_days": 60 }

PASS R6

{ "invoice.supplier_tax_id": "A46990201", "invoice.iban": "ES3520385778983000765410", "invoice.order_reference": "PO-2026-0493", "invoice.total": "12874.4", "invoice.vat_amount": "2234.4", "invoice.issue_date": "2026-06-14" }

norma v3 · erp_dc_d3168b2e349a59c10 · maestro_ev_77bf031266f3bee58

The call on the left, the file the answer comes from on the right. Inés says the invoice is 12,874.40 against an order of 12,847.40, a difference of 27 euros, because that is what the engine found.

We chose this one because it is the feature that proves the rest of the project. It is a separate process we never built into the engine, running from a frozen export, with no network and no credentials during the call. It can hold a conversation about a payment only because the decision and its evidence were stored with enough structure for something external to pick them up and use them. A screenshot cannot fake that.

What it refuses to say is the better half. Faced with an invoice whose IBAN does not match the master, Inés has both numbers in front of her and knows which one is right. She reads out neither. The caller is not verified, and that is exactly the data an account-change fraud needs. She sends him back to the channel that was already verified. A test walks every file to prove it, so the policy fails in pytest if anyone breaks it.

Albertitos plan · la_interseccion · 3

She also never says PAGAR, NO_PAGAR or ESCALAR. Those are internal words, and the engine never authorises a payment by itself, so promising one on the phone would be a lie with a voice.